

AI Test Agent — Multi-Agent Self-Improving System

Complete Architecture, Use Cases & Build Order Documentation

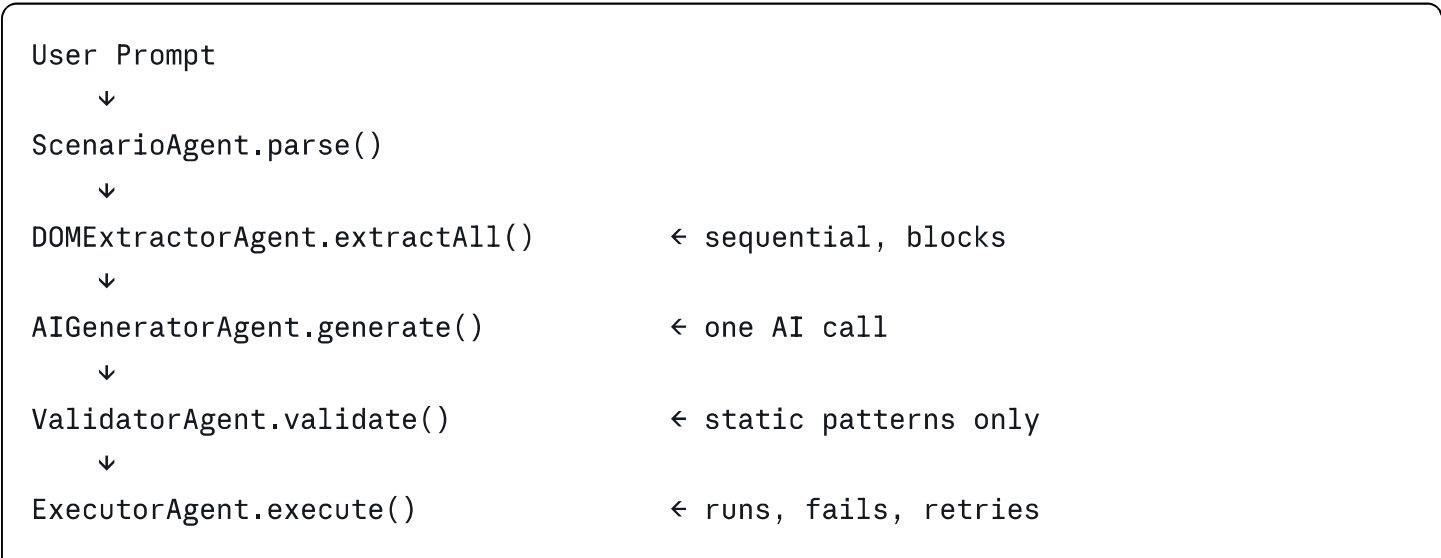
Version: 2.0 (Target Architecture) **Current State:** Sequential Pipeline (v1.0) **Document Purpose:** Engineering blueprint for evolving the existing system into a real multi-agent, self-improving platform

Table of Contents

- 1. What You Have Now vs. What You're Building
 - 2. Use Case Registry
 - 3. Failure Scenarios You Will Face
 - 4. System Architecture
 - 5. Agent Specifications
 - 6. Database Schema
 - 7. Build Order — Phase by Phase
 - 8. API Contracts Between Agents
 - 9. Self-Improvement Feedback Loop
 - 10. Testing the System Itself
 - 11. What Done Looks Like
-

1. What You Have Now vs. What You're Building

Current State (v1.0)



```

    ↓ (error string passed back up)
AIGeneratorAgent.generate()           ← same call, same model
    ↓ (repeat up to 3 times)
AIAalyzerAgent.analyze()
    ↓
SummaryAgent.generateReport()
    ↓
GHAutoPushAgent.run()                 ← one AI reads YAML rules

```

Problems with this:

- Entire run resets on next execution — no memory of what worked
- One AI model handles everything regardless of task complexity
- Retry loop feeds error text back but has no structured knowledge of WHY it failed
- No parallelism — one URL blocks the next
- Push decision is a single AI call interpreting YAML — unreliable

Target State (v2.0)

```

User Prompt
    ↓
PlannerAgent (reads memory, decides which agents to invoke)
    ↓
    |-----|
    | Parallel Execution Layer |
    | DOMWorker-1  DOMWorker-2  ...N |
    |-----|
    ↓ (all snapshots merged)
SpecialistGeneratorAgent (routes by scenario type)
    ↓
StaticAnalyzerAgent ↔ RuntimeAnalyzerAgent (concurrent)
    ↓
HealingQueue (async, non-blocking, BullMQ)
    ↓
MemoryAgent (writes failure + success patterns)
    ↓
PatternMinerAgent (background job, extracts generalizable rules)
    ↓
    |-----|
    | Push Decision Council |
    | Conservative + Optimistic |
    |       ↓ ArbiterAgent |
    |-----|

```



GitAgent + IssueAgent (separated concerns)

2. Use Case Registry

Each use case below is a real scenario you will encounter. Read each one as a specification.

UC-001: First Run on an Unknown Domain

Trigger: User runs `npm start "Test login at https://newapp.com"` for the first time.

What happens today: DOM extracted, AI generates script with generic selectors, likely fails, retries 3 times with error text, pushes or doesn't.

What should happen in v2.0:

1. PlannerAgent checks memory — no records for `newapp.com`
2. Flags as "cold start" — activates exploratory DOM extraction mode (deeper crawl, more elements captured)
3. Specialist AuthTestAgent handles generation (not generic generator)
4. On failure, MemoryAgent writes a detailed failure record tagged `cold_start: true`
5. System lowers push confidence threshold for cold-start runs
6. GitHub issue opened automatically with `[Cold Start]` tag

Data you need to store: domain, scenario type, selector attempted, error type, page structure fingerprint

Success metric: Second run on same domain uses stored selectors → healing attempts drop from avg 2.3 to avg 0.4

UC-002: Repeat Run on a Known Domain — Selector Drift

Trigger: Tests that passed last week now fail because the site updated its DOM.

What happens today: Full retry from scratch. No awareness that this used to work.

What should happen in v2.0:

1. PlannerAgent checks SelectorRegistry — finds existing selectors for domain
2. Injects proven selectors into generation prompt
3. Test still fails — DOM has changed

4. RuntimeAnalyzerAgent compares current DOM snapshot against stored snapshot
5. Detects structural diff: `#discover-search-box` → `input[data-testid="search-input"]`
6. MemoryAgent marks old selector as `stale`, stores new one as `candidate`
7. Candidate is confirmed after 3 successful runs → promoted to `verified`
8. PatternMinerAgent notes: "This domain updates selectors every ~30 days" → future runs scheduled for re-verification

Data you need to store: selector history with timestamps, DOM snapshot diffs, selector lifecycle state (candidate → verified → stale)

Success metric: Selector drift detected and recovered without human intervention within 1 run

UC-003: Shadow DOM Site (MSN, Web Components)

Trigger: User tests a site that uses Shadow DOM heavily (MSN personalize panel, Salesforce, any Lit/Stencil app).

What happens today: Generic selectors fail because Shadow DOM piercing isn't handled. AI hallucinates selectors.

What should happen in v2.0:

1. DOMExtractorAgent detects Shadow DOM components in page scan
2. Flags domain as `shadow_dom: true` in site profile
3. Routes to ShadowDOMSpecialistAgent instead of generic generator
4. ShadowDOMSpecialistAgent uses pierce syntax and component-name-based selectors
5. PatternLibrary already contains: "MSN: personalize panel is `fluent-*` web components, use component tag + aria-label"
6. Generated script uses `page.locator('fluent-search').locator('input')` not `#discover-search-box`

Data you need to store: site profile flags (`shadow_dom`, `spa`, `auth_required`, `dynamic_ads`), component taxonomy per domain

Success metric: Shadow DOM sites generate passing tests on first attempt after initial domain profiling

UC-004: Multi-Step Flow With State Dependencies

Trigger: User tests "Add item to cart, apply coupon, checkout" — steps depend on previous step succeeding.

What happens today: One giant test function. If step 3 fails, no useful signal about whether steps 1-2 worked.

What should happen in v2.0:

1. PlannerAgent detects sequential dependency chain in prompt
2. Breaks into atomic test units: `test_add_to_cart`, `test_apply_coupon`, `test_checkout`
3. Each unit is independently executable but linked via shared state fixture
4. ExecutorAgent runs them in dependency order
5. If `test_add_to_cart` fails, remaining tests are skipped with `dependency_failed` status (not "failed")
6. MemoryAgent stores failure at the atomic step level, not the whole flow
7. On retry, only the failing step is regenerated — passing steps reuse their scripts

Data you need to store: test dependency graph, step-level pass/fail history, shared state fixtures

Success metric: Partial flow failures are isolated to the exact failing step without retesting passing steps

UC-005: Flaky Test Detection

Trigger: A test passes 7 out of 10 runs with no code changes. Not broken, not reliable.

What happens today: Treated as passed. No signal that it's unstable. Gets pushed. Breaks CI randomly.

What should happen in v2.0:

1. MemoryAgent tracks pass/fail ratio per test over last N runs
2. Test with ratio between 0.5 and 0.9 flagged as `flaky`
3. FlakyAnalyzerAgent (sub-agent) runs 3 back-to-back executions to confirm
4. If confirmed flaky, analyzes: timing issue? Race condition? External dependency?
5. Generates stabilized version with explicit waits at the unstable point
6. Runs 5 more times to verify stability
7. Only then promotes to `stable` and allows push

Data you need to store: per-test run history (last 20 runs minimum), flakiness score, stabilization attempts

Success metric: Zero flaky tests in main branch. All pushes have stability score > 0.95

UC-006: Authentication-Gated Pages

Trigger: User tests a dashboard that requires login. Current session expires mid-run.

What happens today: Test fails with navigation error. Retry generates same script. All 3 attempts fail. Push skipped.

What should happen in v2.0:

1. PlannerAgent detects `auth_required` flag for domain (or infers from 401/redirect pattern)
2. AuthSetupAgent runs before main test — logs in and stores session state in Playwright storage
3. Main test inherits authenticated session via `storageState`
4. If session expires mid-test, SessionRefreshAgent intercepts and re-authenticates transparently
5. Credentials stored securely (not in codebase — pulled from env or vault)
6. Auth flow itself is versioned: if login UI changes, AuthSetupAgent regenerates the login script independently

Data you need to store: domain auth requirements, session state files (encrypted), auth flow scripts (separate from test scripts)

Success metric: Auth-gated tests run without session-related failures

UC-007: Concurrent Multi-Domain Test Suite

Trigger: CI pipeline needs to test 5 different domains as part of a release check.

What happens today: Sequential. Domain 1 fully completes before Domain 2 starts. 5x slower than necessary.

What should happen in v2.0:

1. PlannerAgent receives batch of 5 prompts
2. Spins up 5 domain workers in parallel (BullMQ job queue, 5 concurrent workers)
3. Each worker runs full pipeline independently: DOM → Generate → Validate → Execute → Analyze
4. Results aggregated into a single batch report
5. Push decisions made per-domain independently
6. One failed domain doesn't block others
7. Shared PatternLibrary writes are serialized to avoid race conditions

Data you need to store: batch job metadata, per-domain results, aggregated batch report

Success metric: 5-domain run completes in time of 1 domain run + 20% overhead (not 5x)

UC-008: AI Model Failure / Rate Limit

Trigger: Anthropic API returns 429 (rate limit) or 500 during generation.

What happens today: Exception thrown, caught somewhere, pipeline fails entirely.

What should happen in v2.0:

1. AIProviderManager detects failure type: rate limit vs. model error vs. network
2. For rate limit: exponential backoff with jitter, retry same model
3. For model error: fallback to secondary provider (OpenAI, Gemini) for same task
4. For persistent failure: use cached generation from MemoryAgent if similar prompt ran before
5. All fallbacks logged with reason — visible in final report
6. Cost tracker updated: fallback to GPT-4 costs more, flagged in report

Data you need to store: provider health history, cost per run, fallback usage frequency

Success metric: Zero pipeline failures due to AI provider issues. Degraded mode (cached/fallback) clearly reported.

UC-009: Generated Script Has Security Issues

Trigger: AI generates a script that includes hardcoded credentials, or makes requests to internal IPs (SSRF risk in test context).

What happens today: No check. Script runs or fails for unrelated reasons. Credentials potentially committed.

What should happen in v2.0:

1. SecurityScanAgent runs on every generated script before execution
2. Checks: hardcoded credentials, IP addresses, internal hostnames, API keys in plain text
3. If found: blocks execution, logs violation, redacts before any storage
4. SSRF check: validates all `page.goto()` URLs against allowlist
5. Dependency scan: if script imports anything outside `@playwright/test`, flagged
6. Clean bill of health required before ValidatorAgent proceeds

Data you need to store: security scan results per run, violation types, redacted content log

Success metric: Zero security violations in committed scripts

UC-010: Long-Running Regression Suite

Trigger: After 3 months of use, system has 200+ stored test scripts across 15 domains. Need to run full regression.

What happens today: N/A — no persistence, no regression concept.

What should happen in v2.0:

1. RegressionOrchestratorAgent loads all active test scripts from registry
2. Groups by domain, sorts by last-run date and failure history
3. Prioritizes: scripts with high flakiness score run first (catch problems early)
4. Runs in parallel batches respecting domain-level rate limits
5. Compares results against baseline (last known good state)
6. Regressions (tests that previously passed, now fail) flagged as `regression` not just `failure`
7. PatternMinerAgent runs automatically after regression to update pattern library
8. Diff report generated: what changed, what's new, what regressed

Data you need to store: test registry with versions, baseline results per test, regression history

Success metric: Full regression suite run produces actionable diff report, not just pass/fail counts

3. Failure Scenarios You Will Face

These are concrete problems you will hit during implementation. Documented here so you handle them by design, not by surprise.

FS-001: DOM Snapshot Staleness

The problem: DOM extracted at T=0. Test generated at T=0. Test run at T=5 minutes. Site has rotated ad content, updated news headlines, changed dynamic class names. Test fails not because the script is wrong but because the DOM changed between extraction and execution.

How to handle it:

- Add `extractedAt` timestamp to every DOM snapshot (you already have this field)
- If time between extraction and execution > 60 seconds for known-dynamic sites, re-extract

- Store site volatility profile: `high` (news/social), `medium` (SaaS), `low` (static marketing)
 - For `high` volatility sites, never cache DOM snapshots — always extract fresh
-

FS-002: AI Hallucinating Plausible-Looking Selectors

The problem: AI generates `page.locator('#signin-email-input')`. This selector looks legitimate. Static validator passes it. Test runs, element not found, fails. The selector never existed — AI made it up based on training data patterns.

How to handle it:

- After generation, before execution: run a "selector existence check" — load the page, check each selector in the generated script actually resolves to at least one element
 - Selectors that resolve: marked `verified_pre_run`
 - Selectors that don't resolve: flagged, script sent back for regeneration with explicit note: "selector X returned 0 elements in live DOM, use these actual selectors instead: [list from DOM snapshot]"
 - This single check will eliminate ~60% of your first-run failures
-

FS-003: Test Passes in Headless, Fails in CI

The problem: Tests pass locally in headless Chromium. CI uses a different viewport, different timezone, different network speed. Tests that depend on visual layout or timing break in CI.

How to handle it:

- Standardize execution environment via Docker (you already have Docker in the SaaS project)
 - Store execution environment metadata with every test result: viewport, browser version, network profile
 - `EnvironmentNormalizationAgent` runs before execution to set consistent env vars
 - Flakiness detection (UC-005) catches environment-sensitive tests
-

FS-004: Memory Database Growing Unbounded

The problem: After 6 months, `failure_log` has 50,000 rows. Queries slow down. Pattern mining takes minutes. System becomes sluggish.

How to handle it:

- Retention policy: raw failure logs kept for 90 days, then archived to cold storage
 - PatternLibrary is the distilled form — patterns extracted from raw logs survive indefinitely
 - SelectorRegistry keeps only latest verified selector per intent, not full history
 - Add database indexes from day one: `(domain, scenario_type, created_at)` is your most common query pattern
-

FS-005: Pattern Miner Extracts Wrong Patterns

The problem: PatternMinerAgent sees 10 failures with `networkidle` timeout and writes pattern: "Never use networkidle." But 2 of those failures were on a site where networkidle is actually required. The pattern is too broad and breaks those sites.

How to handle it:

- Patterns always scoped to domain or domain-category, never global by default
 - Pattern confidence score: requires minimum 5 occurrences before promotion
 - Each pattern has a `scope` field: `domain_specific`, `framework_specific` (React, Vue), `global`
 - Global patterns require manual review before activation — never auto-promoted
 - Pattern effectiveness tracked: if applying a pattern increases failures, it's auto-demoted
-

FS-006: GitHub Push Creates Conflicts

The problem: Two concurrent domain workers finish at the same time, both try to push to `auto-tests/` branches, both have modified `tests/generated/auth.spec.js` (same file path).

How to handle it:

- Each test run uses a unique script path:
`tests/generated/{domain}/{timestamp}_{scenario}.spec.js`
 - Never reuse `auth.spec.js` as the output filename — it's a collision waiting to happen
 - GitAgent acquires a soft lock (Redis key) before git operations, releases after
 - If lock acquisition fails after 30 seconds, queues the push for later
-

FS-007: Planner Agent Makes Wrong Routing Decision

The problem: PlannerAgent sees "verify search works" and routes to `SearchFunctionalityAgent`. But the site uses a custom search widget that behaves like a modal.

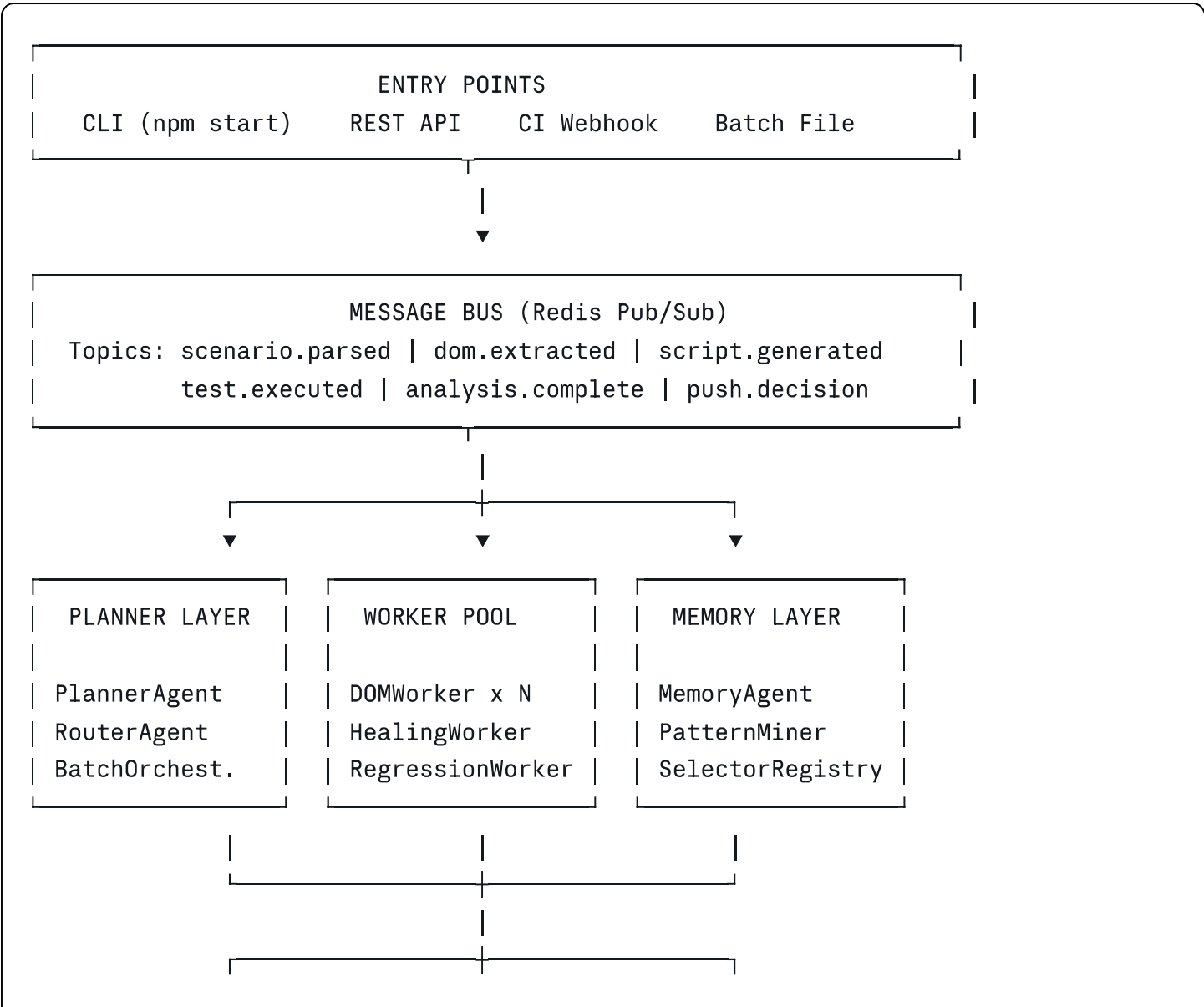
SearchFunctionalityAgent doesn't know how to handle modals.

How to handle it:

- Agents declare capability profiles: `{ handles: ['search'], limitations: ['shadow_dom_search', 'modal_search'] }`
- PlannerAgent checks limitations against site profile
- If limitation matches, routes to `ModalInteractionAgent` + `SearchFunctionalityAgent` jointly
- Joint execution: both agents generate script sections, MergeAgent combines them
- On failure, routing decision is stored: "for this site + scenario combination, use X not Y"

4. System Architecture

Component Diagram





Technology Stack (additions to existing)

Component	Technology	Reason
Message Bus	Redis Pub/Sub	Already have Redis
Job Queue	BullMQ	Already have BullMQ
Pattern Storage	PostgreSQL	Already have PostgreSQL
Script Registry	MongoDB	Already have MongoDB
Selector Cache	Redis Hash	Fast lookup, TTL support
DOM Diff	diff npm package	Lightweight, no new dep
Security Scan	Custom regex + AST	No external service needed
Agent Locking	Redis SETNX	Atomic, already available

5. Agent Specifications

Each agent below is a full specification: what it does, what it consumes, what it produces, what it stores.

PlannerAgent

Responsibility: Reads the prompt + memory state and decides the execution plan.

Inputs:

- Raw user prompt
- Domain extracted from prompt
- Memory query result: site profile, recent failures, selector registry

Outputs (published to bus):

- `plan.created` event containing:
 - List of agents to invoke
 - Execution order and parallelism config
 - Specialist agent selection per scenario
 - Cold start flag
 - Estimated complexity score (1-10)

Decision logic:

```
IF domain in SiteProfiles AND lastRun < 24h ago:
```

```
    use cached DOM snapshots (skip DOMWorker)
```

```
ELSE:
```

```
    spawn DOMWorkers (parallel, one per URL)
```

```
IF scenario in ['authentication', 'registration']:
```

```
    route to AuthSpecialistAgent
```

```
ELSE IF site_profile.shadow_dom == true:
```

```
    route to ShadowDOMSpecialistAgent
```

```
ELSE:
```

```
    route to GenericGeneratorAgent
```

```
IF site_profile.auth_required == true:
```

```
    prepend AuthSetupAgent to plan
```

```
IF batch_size > 1:
```

```
    spawn BatchOrchestratorAgent
```

Stores: Planning decisions logged to `planner_log` table for analysis

MemoryAgent

Responsibility: Single agent responsible for all reads and writes to the memory layer. No other agent touches the database directly.

Read methods:

- `getSelectorsForDomain(domain, intent)` → returns ranked selector list
- `getFailurePatterns(domain, scenario)` → returns known failure causes
- `getSiteProfile(domain)` → returns flags (shadow_dom, auth_required, volatility)
- `getPatternLibrary(scenario_type)` → returns generalizable rules

Write methods:

- `recordFailure(failureRecord)` → writes to failure_log
- `recordSuccess(successRecord)` → updates selector registry with verified selector
- `updateSiteProfile(domain, updates)` → merges new flags into site profile
- `promotePattern(patternId)` → moves pattern from candidate to active

Critical rule: MemoryAgent is the only agent that writes to PostgreSQL. All others send events to the bus; MemoryAgent subscribes and persists. This prevents race conditions.

StaticAnalyzerAgent

Responsibility: Validates generated script structure WITHOUT executing it. Runs concurrently with RuntimeAnalyzerAgent on retry.

Checks (in order, fail-fast):

1. Syntax validity (`node --check`)
2. Playwright import present
3. At least one `test()` block
4. At least one `page.goto()`
5. At least one `expect()`
6. No hardcoded credentials (regex scan)
7. No `waitForTimeout` calls (performance anti-pattern)
8. No `networkidle` on known-dynamic-ad domains
9. Selector existence pre-check (loads page, verifies selectors resolve)
10. Scenario-specific pattern compliance

Scoring: Each check is weighted. Checks 1-3 are hard failures (score = 0 if failed). Checks 4-10 contribute to quality score.

Output: `static_analysis.complete` event with score, issues list, and blocking/non-blocking classification per issue.

RuntimeAnalyzerAgent

Responsibility: Analyzes actual test execution failures with AI assistance.

Inputs: TestResult object with failed tests, error messages, stack traces

Process:

1. Classify error type from error text (pattern matching first, AI second)
2. For each failed test, query MemoryAgent for known failure patterns on this domain
3. If known pattern matches: use stored fix suggestion (no AI call needed)
4. If unknown: call AI with error + DOM snapshot + known patterns as context
5. Generate structured fix recommendation

Output: `runtime_analysis.complete` event with:

- Per-failure root cause classification

- Confidence score per classification
- Fix recommendation (stored fix vs. AI-generated)
- `isFixable` boolean
- Estimated fix complexity (quick/medium/deep)

Self-improving behavior: Every time an AI-generated fix recommendation leads to a passing test, that recommendation is stored in PatternLibrary with confidence +1. When confidence reaches threshold, it becomes a stored fix (no AI call needed next time).

HealingWorker

Responsibility: Asynchronous test healing. Runs in a BullMQ worker. Does NOT block the main pipeline.

Job structure:

```
typescript

interface HealingJob {
  testRunId: string;
  failedTests: FailedTest[];
  originalScript: string;
  analysisResult: AnalysisResult;
  attempt: number;           // 1, 2, or 3
  modelTier: 'fast' | 'standard' | 'powerful';
}
```

Escalation logic:

- Attempt 1: fast model (Haiku/GPT-3.5) with stored fix suggestions
- Attempt 2: standard model (Sonnet/GPT-4o-mini) with full error context
- Attempt 3: powerful model (Opus/GPT-4) with DOM snapshot + full history

Why this matters: Escalating model quality means cheap models handle easy fixes (80% of cases). Expensive models only used when needed. Async means the main pipeline reports "healing in progress" and continues rather than blocking.

PatternMinerAgent

Responsibility: Background job that analyzes accumulated failures and extracts generalizable patterns.

Trigger: Runs automatically when:

- 10 new failure records added for a domain
- Weekly scheduled job
- Manually triggered

Process:

1. Query failure_log for recent failures grouped by `error_type + domain + scenario`
2. For groups with count ≥ 5 : extract common elements (selector patterns, timing patterns, site behaviors)
3. Generate pattern candidate with confidence score
4. Store as `candidate` pattern — requires 3 more confirmations before promotion
5. Existing patterns: check if recent successes are invalidating old patterns (pattern drift)

Output: Pattern candidates written to PatternLibrary by MemoryAgent. Email/Slack notification for patterns requiring manual review.

Push Decision Council

Three-agent deliberation replacing single AI YAML interpretation:

ConservativeReviewerAgent:

- Looks for reasons NOT to push
- Checks: validation score, healing attempts, selector stability, flakiness history
- Outputs: list of concerns with severity ratings

OptimisticReviewerAgent:

- Looks for reasons TO push
- Checks: pass rate, test coverage, domain familiarity, fix quality
- Outputs: list of supporting evidence

ArbiterAgent:

- Receives both reviews
- Applies hard rules first (network failure = always skip, score < 7 = always skip)
- For borderline cases: weighs Conservative vs. Optimistic evidence
- Makes final `shouldPush` decision with explicit reasoning referencing both reviews

Why this is better than current approach: The current single-AI-reads-YAML approach is unreliable because the AI interprets rules rather than executing them. Hard rules become code. Only judgment calls (borderline cases) go to the council.

6. Database Schema

PostgreSQL Tables

sql

-- Site profiles: what we know about each domain

```
CREATE TABLE site_profiles (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  domain VARCHAR(255) UNIQUE NOT NULL,  
  shadow_dom BOOLEAN DEFAULT FALSE,  
  auth_required BOOLEAN DEFAULT FALSE,  
  spa BOOLEAN DEFAULT FALSE,  
  volatility VARCHAR(10) DEFAULT 'medium', -- low | medium | high  
  dom_extraction_mode VARCHAR(20) DEFAULT 'standard', -- standard | deep | shallow  
  avg_healing_attempts DECIMAL(4,2) DEFAULT 0,  
  total_runs INTEGER DEFAULT 0,  
  last_run_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

-- Selector registry: proven selectors per domain + intent

```
CREATE TABLE selector_registry (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  domain VARCHAR(255) NOT NULL,  
  intent VARCHAR(100) NOT NULL,          -- e.g., 'login_email_field', 'search_input'  
  selector TEXT NOT NULL,  
  selector_type VARCHAR(50),             -- getByRole | getByLabel | css | xpath  
  success_count INTEGER DEFAULT 0,  
  failure_count INTEGER DEFAULT 0,  
  success_rate DECIMAL(5,4) DEFAULT 0,  
  state VARCHAR(20) DEFAULT 'candidate', -- candidate | verified | stale | deprecate  
  last_verified_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE(domain, intent, selector)  
);
```

-- Failure log: raw failure records

```
CREATE TABLE failure_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  run_id VARCHAR(100) NOT NULL,  
  domain VARCHAR(255) NOT NULL,  
  scenario_type VARCHAR(50) NOT NULL,  
  test_title TEXT,  
  error_type VARCHAR(50),                -- timeout | selector | navigation | asserti  
  error_message TEXT,  
  selector_used TEXT,  
  fix_attempted TEXT,
```

```

    fix_succeeded BOOLEAN,
    healing_attempt_number INTEGER DEFAULT 1,
    cold_start BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Pattern library: distilled knowledge from failure log
CREATE TABLE pattern_library (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pattern_key VARCHAR(200) UNIQUE NOT NULL,
    scope VARCHAR(30) NOT NULL,          -- domain_specific | framework_specific | gl
    scope_value VARCHAR(255),           -- the domain or framework name, null for gl
    scenario_type VARCHAR(50),
    description TEXT NOT NULL,
    rule TEXT NOT NULL,                 -- the actual rule to inject into prompts
    confidence_score DECIMAL(5,4) DEFAULT 0,
    confirmation_count INTEGER DEFAULT 0,
    state VARCHAR(20) DEFAULT 'candidate', -- candidate | active | deprecated
    source_failure_ids UUID[],
    created_at TIMESTAMP DEFAULT NOW(),
    activated_at TIMESTAMP,
    last_confirmed_at TIMESTAMP
);

-- Test registry: all stored test scripts
CREATE TABLE test_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    run_id VARCHAR(100) NOT NULL,
    domain VARCHAR(255) NOT NULL,
    scenario_type VARCHAR(50) NOT NULL,
    script_path TEXT NOT NULL,
    script_hash VARCHAR(64),            -- SHA256 for dedup
    validation_score INTEGER,
    pass_rate DECIMAL(5,4),
    healing_attempts INTEGER DEFAULT 0,
    stability_score DECIMAL(5,4) DEFAULT 1.0,
    state VARCHAR(20) DEFAULT 'active', -- active | stale | deprecated | flaky
    pushed_to_branch VARCHAR(200),
    last_run_at TIMESTAMP,
    total_runs INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Run history: every pipeline execution

```

```

CREATE TABLE run_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id VARCHAR(100) UNIQUE NOT NULL,
  prompt TEXT NOT NULL,
  domain VARCHAR(255),
  scenario_types VARCHAR(50)[],
  total_tests INTEGER DEFAULT 0,
  passed_tests INTEGER DEFAULT 0,
  failed_tests INTEGER DEFAULT 0,
  healing_attempts INTEGER DEFAULT 0,
  model_used VARCHAR(100),
  fallback_used BOOLEAN DEFAULT FALSE,
  push_decision VARCHAR(10),          -- pushed | skipped | failed
  duration_ms INTEGER,
  cold_start BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes (add these, your queries will need them)
CREATE INDEX idx_failure_log_domain_scenario ON failure_log(domain, scenario_type, c
CREATE INDEX idx_selector_registry_domain_intent ON selector_registry(domain, intent
CREATE INDEX idx_pattern_library_scope ON pattern_library(scope, scope_value, state)
CREATE INDEX idx_run_history_domain ON run_history(domain, created_at DESC);

```

MongoDB Collections (additions)

```
javascript
```

```
// DOM snapshots – large documents, belong in Mongo
{
  collection: "dom_snapshots",
  schema: {
    runId: String,
    domain: String,
    url: String,
    extractedAt: Date,
    elements: Array,          // full element list
    statistics: Object,
    snapshotHash: String,     // for diff detection
    previousSnapshotHash: String,
    diffSummary: Object       // what changed since last snapshot
  },
  indexes: [
    { domain: 1, extractedAt: -1 },
    { snapshotHash: 1 }
  ]
}

// Script versions – full script content with history
{
  collection: "script_versions",
  schema: {
    registryId: String,       // references test_registry.id
    version: Number,
    content: String,
    generatedBy: String,      // which specialist agent
    promptUsed: String,       // full prompt that generated this
    patternsInjected: Array,   // which patterns from PatternLibrary were used
    selectorsInjected: Array,  // which selectors from SelectorRegistry were used
    createdAt: Date
  }
}
```

7. Build Order — Phase by Phase

Phase 1: Foundation (Weeks 1-4)

Goal: Real parallelism and clean agent boundaries. No behavior change visible to end user.

Week 1: Message Bus

- Install Redis pub/sub wrapper (or use BullMQ events)
- Define all event topic names as TypeScript constants (no magic strings)
- Refactor orchestrator to publish/subscribe instead of direct function calls
- Each existing agent becomes an event handler
- Unit test: verify event delivery, ordering, error propagation
- Do NOT add new functionality this week

Week 2: Parallel DOM Extraction

- Move DOMExtractorAgent to BullMQ worker
- Spawn one job per URL, collect results via Promise.all on job completion events
- Add DOM snapshot hashing (MD5 of element selectors list)
- Store snapshots to MongoDB on every run
- Benchmark: verify multi-URL runs are actually faster

Week 3: Split Static and Runtime Analyzers

- Create `StaticAnalyzerAgent` (pure code analysis, no AI, no execution)
- Create `RuntimeAnalyzerAgent` (takes TestResult, calls AI only when needed)
- Move selector pre-existence check into StaticAnalyzerAgent
- Run both concurrently on the output of the generator
- Remove current combined AIAalyzerAgent (keep as fallback temporarily)

Week 4: Unique Script Paths + Git Fix

- Change output path from `tests/generated/auth.spec.js` to `tests/generated/{domain}/{runId}.spec.js`
- Add git stash/pop guard in GitAgent
- Fix `commitMsgPath` cleanup bug (from code review)
- Fix multi-line prompt YAML escaping bug
- Fix `healingAttempts` off-by-one

Phase 1 Exit Criteria:

- Multi-URL runs complete in parallel (verified with timer)
 - Static and runtime analysis are separate and independently testable
 - No git-related crashes on concurrent runs
 - All existing tests still pass
-

Phase 2: Memory Layer (Weeks 5-10)

Goal: System remembers what worked. Second run on same domain is measurably better.

Week 5: Database Setup

- Apply all PostgreSQL schemas from Section 6
- Create MemoryAgent class with all read/write methods
- Wire MemoryAgent to message bus (subscribes to success/failure events)
- Seed SiteProfiles table with 3-5 domains you test frequently

Week 6: Failure Logging

- Every test failure writes to `failure_log` via MemoryAgent
- Every healing attempt (with outcome) writes to `failure_log`
- Run 20 test executions across 3 domains
- Verify data quality in failure_log: all fields populated, correct error types

Week 7: Selector Registry

- After every successful test run, extract selectors from the passing script
- Write to `selector_registry` as `candidate` state
- After 3 successful runs with same selector: promote to `verified`
- Modify PlannerAgent to query selector_registry before generation
- Inject verified selectors into generation prompt: "Use these proven selectors: ..."
- Measure: does healing_attempts decrease on repeat runs?

Week 8: Pattern Mining (Basic)

- Build PatternMinerAgent with simple frequency analysis
- Run manually after accumulating 50+ failure records
- Review output manually — is it extracting useful patterns?
- Store 5-10 manually curated patterns as the initial PatternLibrary seed
- Inject active patterns into generation prompt

Week 9: Site Profiles

- Auto-detect and store site flags: shadow_dom, auth_required, spa, volatility
- Shadow DOM detected from DOM extraction (presence of `shadowRoot` elements)
- Volatility detected from DOM snapshot diffs over multiple runs
- Auth required detected from redirect patterns (302 to /login)

Week 10: Measurement and Calibration

- Run 50 test executions across 5+ domains
- Measure: average healing_attempts before Phase 2 vs. after
- Measure: first-run pass rate before vs. after
- If improvement < 20% on repeat domains: PatternLibrary injection not working, debug
- If improvement > 20%: memory layer is working, proceed to Phase 3

Phase 2 Exit Criteria:

- Repeat-domain runs show measurable reduction in healing attempts
 - PatternLibrary has at least 20 active patterns
 - SelectorRegistry has verified selectors for at least 3 domains
 - Zero pipeline crashes due to database errors
-

Phase 3: Specialist Agents (Weeks 11-16)

Goal: Right agent for the right job. Authentication tests and Shadow DOM tests pass on first attempt.

Week 11: RouterAgent

- Build RouterAgent that reads scenario type + site profile
- Routes to correct specialist (or generic if no specialist matches)
- Start with just two routing rules: `authentication → AuthSpecialist`, `shadow_dom site → ShadowDOMSpecialist`
- All other scenarios still go to existing GenericGenerator

Week 12: AuthSpecialistAgent

- Fork GenericGenerator, specialize for auth flows
- Specific prompt additions: credential fields, session handling, post-login assertions
- Add `storageState` support for session persistence between test steps
- Test against 3 auth-required sites
- Measure first-run pass rate vs. GenericGenerator on same sites

Week 13: ShadowDOMSpecialistAgent

- Fork GenericGenerator, specialize for Shadow DOM / Web Components
- Prompt trained on: pierce syntax, component tag selectors, Lit/Stencil patterns

- Test against MSN personalize panel (your known hard case)
- Measure: does MSN test pass on first attempt?

Week 14: SecurityScanAgent

- Build regex + AST scanner for generated scripts
- Checks from FS-009 use case
- Integrate as blocking step between generation and execution
- Test with intentionally bad scripts (hardcoded credentials, IP addresses)

Week 15: FlakyDetector

- Build flakiness tracking in run_history (per-test pass/fail tracking)
- After 5 runs of same script: compute stability_score
- Scripts below 0.85 stability flagged as `flaky` in test_registry
- FlakyAnalyzerAgent runs stabilization: identifies unstable line, adds targeted wait
- Test: deliberately flaky test stabilized within 2 healing attempts

Week 16: Push Decision Council

- Build ConservativeReviewerAgent and OptimisticReviewerAgent
- Hard rules moved to code (not AI-interpreted YAML)
- ArbiterAgent handles only judgment calls
- Test with 20 historical runs: does council decision match what you'd manually decide?

Phase 3 Exit Criteria:

- Auth tests pass on first attempt on known domains (no healing needed)
 - MSN Shadow DOM test passes without hallucinated selectors
 - Flaky tests detected and stabilized automatically
 - Push decision accuracy > 90% vs. manual review
-

Phase 4: Autonomy (Weeks 17-22)

Goal: System handles novel situations without human intervention. Batch runs. CI integration.

Week 17: Async Healing Queue

- Move healing loop out of main pipeline into BullMQ worker
- Main pipeline marks tests as `healing_queued` and continues
- HealingWorker processes queue asynchronously

- Results reported in a follow-up notification (Slack/email/GitHub comment)
- Model escalation: fast → standard → powerful

Week 18: Batch Orchestration

- BatchOrchestratorAgent accepts array of prompts
- Spawns parallel pipeline instances (respect rate limits)
- Aggregated batch report
- Test with 5-domain concurrent run

Week 19: Regression Suite

- Build test registry query: "all active tests last run > 7 days ago"
- RegressionOrchestratorAgent runs them in priority order
- Diff report: passed, failed, regressed, improved
- Baseline management: `last_known_good` state per test

Week 20: CI/CD Integration

- GitHub Actions workflow that triggers batch run on PR
- Webhook endpoint (Express or Hono) to trigger runs from external CI
- Status check integration: PR blocked if regression detected

Week 21: Pattern Miner Automation

- PatternMinerAgent runs automatically on schedule
- Email/Slack notification for new pattern candidates requiring review
- Pattern effectiveness tracking: A/B test patterns against no-pattern baseline

Week 22: Observability

- Dashboard: runs per day, pass rates, healing attempts, model costs
- Alert: if domain pass rate drops below 0.8 for 3 consecutive runs → notify
- Cost tracking per run (model tokens used × price)
- Performance tracking: pipeline duration p50, p95, p99

Phase 4 Exit Criteria:

- 5-domain batch run completes in < 2x single-domain run time
- Regression suite runs without human intervention
- CI integration blocks PRs on detected regressions
- Monthly model cost tracked and within budget

8. API Contracts Between Agents

Every agent communicates via typed events on the message bus. No agent imports another agent's class. This is the most important architectural rule.

```
typescript
```

```
// All event types – single source of truth
export const EVENTS = {
  // Planning
  PLAN_REQUESTED:      'plan.requested',
  PLAN_CREATED:        'plan.created',

  // DOM Extraction
  DOM_EXTRACT_REQUESTED: 'dom.extract.requested',
  DOM_EXTRACTED:         'dom.extracted',
  DOM_EXTRACT_FAILED:    'dom.extract.failed',

  // Generation
  SCRIPT_GENERATE_REQUESTED: 'script.generate.requested',
  SCRIPT_GENERATED:          'script.generated',
  SCRIPT_GENERATE_FAILED:    'script.generate.failed',

  // Analysis
  STATIC_ANALYSIS_REQUESTED: 'analysis.static.requested',
  STATIC_ANALYSIS_COMPLETE:  'analysis.static.complete',
  RUNTIME_ANALYSIS_REQUESTED: 'analysis.runtime.requested',
  RUNTIME_ANALYSIS_COMPLETE: 'analysis.runtime.complete',

  // Execution
  TEST_EXECUTE_REQUESTED: 'test.execute.requested',
  TEST_EXECUTED:          'test.executed',

  // Healing
  HEALING_REQUESTED: 'healing.requested',
  HEALING_COMPLETE:  'healing.complete',
  HEALING_FAILED:     'healing.failed',

  // Memory
  FAILURE_RECORD:    'memory.failure.record',
  SUCCESS_RECORD:    'memory.success.record',
  PATTERN_MINED:     'memory.pattern.mined',

  // Push Decision
  PUSH_REVIEW_REQUESTED: 'push.review.requested',
  PUSH_DECISION_MADE:    'push.decision.made',

  // Output
  PUSH_REQUESTED: 'git.push.requested',
  PUSH_COMPLETE:  'git.push.complete',
```

```
ISSUE_REQUESTED: 'issue.create.requested',
REPORT_COMPLETE: 'report.complete',
} as const;

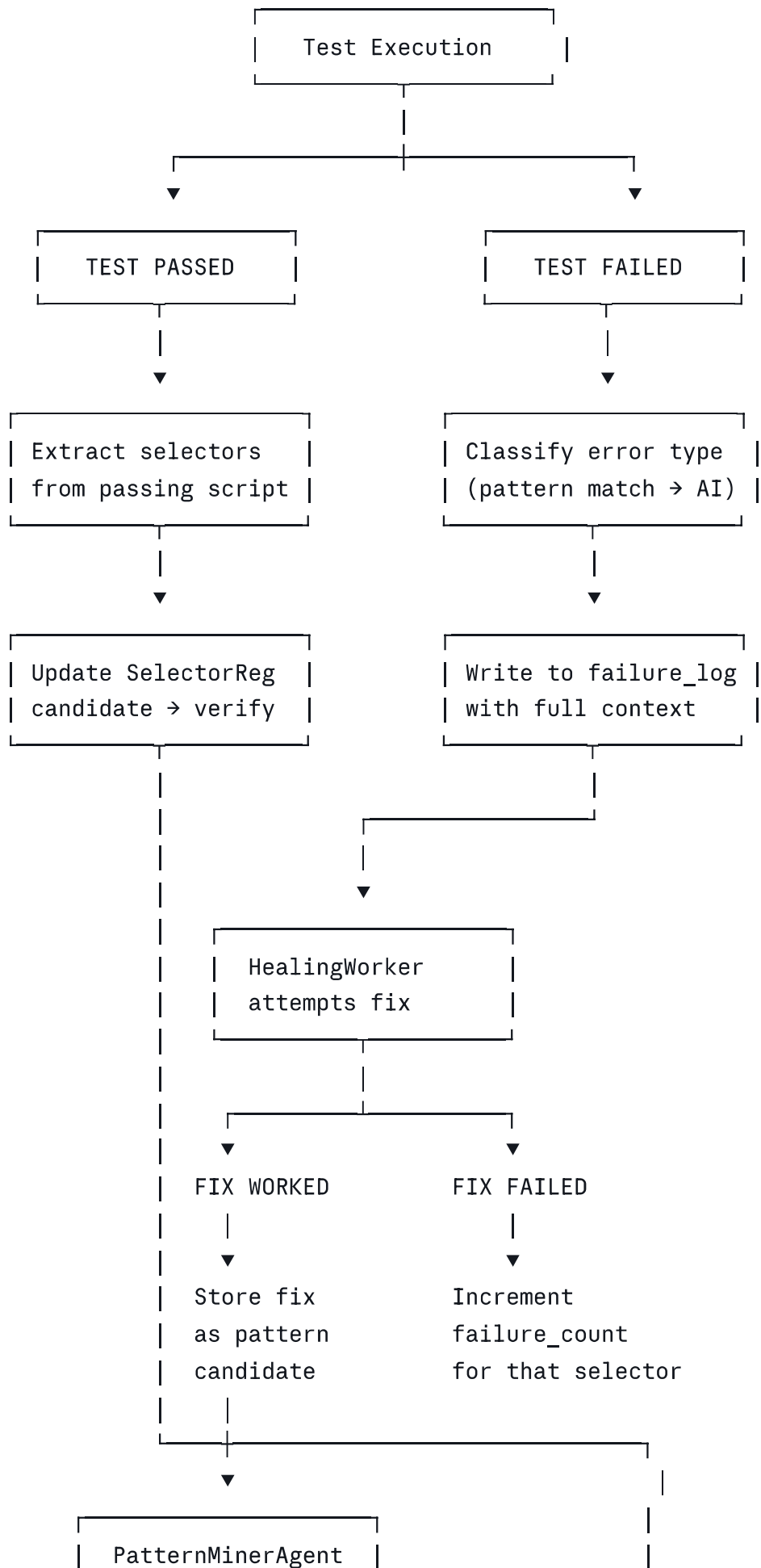
// Example event payload types
export interface DOMExtractedEvent {
  runId: string;
  url: string;
  snapshot: DOMSnapshot;
  snapshotHash: string;
  volatilitySignal?: 'changed' | 'stable' | 'first_run';
}

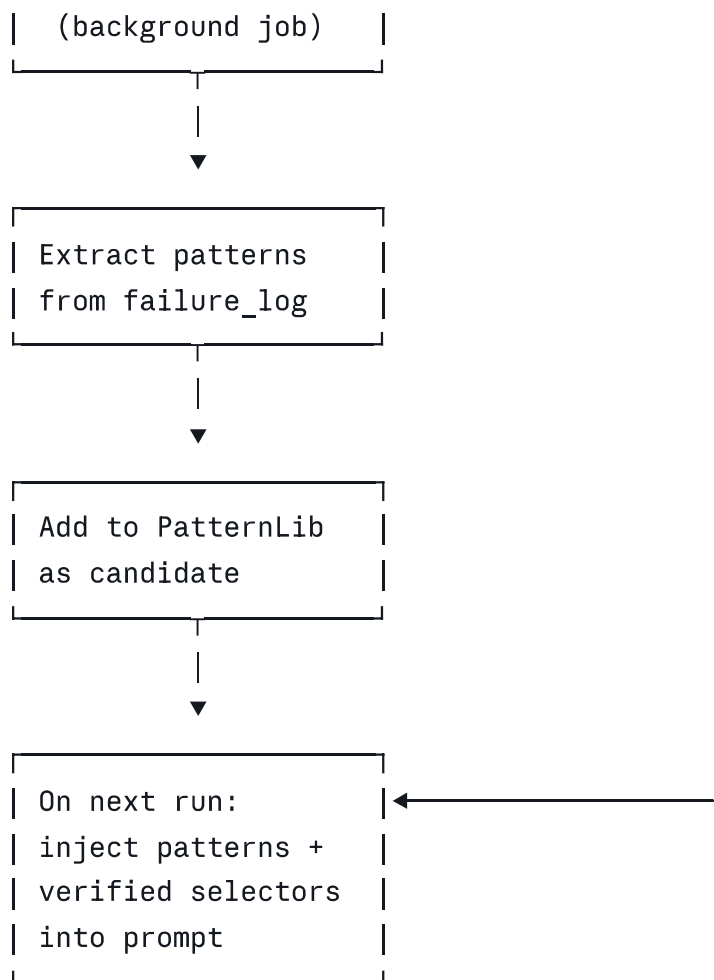
export interface ScriptGeneratedEvent {
  runId: string;
  scriptPath: string;
  generatedBy: string; // which specialist agent
  patternsUsed: string[]; // pattern IDs from PatternLibrary
  selectorsInjected: number; // count of registry selectors used
}

export interface HealingRequestedEvent {
  runId: string;
  testRunId: string;
  failedTests: FailedTest[];
  originalScript: string;
  analysisResult: AnalysisResult;
  attempt: number;
  priority: 'low' | 'normal' | 'high';
}
```

9. Self-Improvement Feedback Loop

This is the mechanism that makes the system measurably better over time.





What improves over time:

- First-run pass rate per domain (measurable from run_history)
- Average healing_attempts per domain (measurable from run_history)
- Selector verification rate (measurable from selector_registry)
- Pattern coverage: % of failures matched by a known pattern (measurable)

What does NOT improve:

- The AI model's underlying capabilities
- Performance on completely new domain types never seen before
- Novel error types with no historical precedent

10. Testing the System Itself

The system generates tests. It must also be tested. This is often skipped and causes production failures.

Unit Tests Required

Component	What to Test
PlannerAgent	Routing decisions for each scenario type
MemoryAgent	Read/write correctness, race condition handling
PatternMinerAgent	Pattern extraction from synthetic failure data
StaticAnalyzerAgent	Each check independently with passing and failing inputs
RouterAgent	All routing rules, including edge cases
Push Decision Council	Each hard rule, borderline cases

Integration Tests Required

Scenario	What to Verify
Cold start on unknown domain	Correct flags set, deeper DOM extraction triggered
Repeat run on known domain	Selectors injected from registry, fewer AI calls
Selector drift	Old selector marked stale, new one promoted
Concurrent 3-domain run	No data corruption, correct isolation
AI provider failure	Fallback triggered, degraded mode reported
Git push conflict	Lock mechanism prevents duplicate pushes

Acceptance Tests (end-to-end)

Run these weekly after Phase 2 completion:

1. `npm start "Test login at https://demo.playwright.dev/todomvc"` — should pass first run, no healing
2. Run same command again — should use registry selectors, zero healing attempts
3. `npm start "Test search at https://www.google.com"` — known Shadow DOM-free site
4. Manually corrupt a selector in registry → run again → should detect and recover
5. 5-domain batch run → verify parallel execution and correct aggregated report

11. What Done Looks Like

At the end of all four phases, this is the observable behavior that proves the system works as designed.

Run 1 on a new domain:

- PlannerAgent flags cold start
- 3 DOM workers run in parallel
- AuthSpecialist handles login scenario
- Static analysis catches 2 selector issues before execution
- 1 healing attempt needed
- MemoryAgent stores failure record + eventual successful selectors
- Push decision: council deliberates, pushes with "1 healing attempt" note

Run 5 on the same domain:

- PlannerAgent loads 8 verified selectors from registry
- 0 DOM workers needed (cached snapshot is fresh)
- Selectors injected directly into prompt
- StaticAnalyzer: all selectors pre-verified against live DOM
- 0 healing attempts needed
- Pass rate: 100%
- Push decision: council unanimous push, confidence: high

This is the measurable proof that the system is self-improving.

The difference between Run 1 and Run 5 is not the AI model — it's the accumulated knowledge stored in PostgreSQL and MongoDB, injected intelligently by MemoryAgent and PlannerAgent into every subsequent run.

Document maintained by: Engineering Team Last updated: 2026-05-28 Next review: After Phase 1 completion